

Короткий контекст

На терминале установлен сервер видеонаблюдения **Macroscop 4.6** с модулем распознавания автономеров (ANPR). Камеры на КПП фиксируют проезд транспорта; распознанный номер и направление движения («Въезд» / «Выезд») доступны через HTTP API Macroscop.

Проблема: сейчас факт приезда/отъезда машины не попадает в Steiza автоматически — диспетчер вынужден вводить данные вручную или не фиксировать их вовсе.

Цель задачи:

1. Реализовать **модуль автопарка** (справочник машин терминала) с CRUD через API Steiza.
2. Реализовать **журнал въездов/выездов**, заполняемый автоматически из событий Macroscop.
3. Реализовать **сервис интеграции** с Macroscop, который получает события распознавания номеров и записывает их в Steiza.

Связанные задачи:

- [T-031](#) — справочник автопарка (частично пересекается; T-068 детализирует и добавляет интеграцию с камерами).
- [T-037](#) — флоу приёма машин на терминале (в перспективе журнал въездов может стать источником для реестра приёма).

Внешние контакты и доступы:

- Оперативные вопросы по серверу терминала: [Telegram @ter224](#).
- Файл VPN-конфигурации (`.conf`) предоставляется отдельно заказчиком.

Документация Macroscop API: [macroscop-4.6-api.pdf](#) (REST API + HTTP API).

Инфраструктура и предварительные проверки

Доступ к серверу терминала

Параметр	Значение
IP (в VPN)	10.66.66.21

Параметр	Значение
Macroscop API	<code>http://10.66.66.21:8080</code>
Удалённый доступ	RDP (через VPN)
RDP-пользователь	User
RDP-пароль	123 <i>(хранить только в секретах, не коммитить)</i>

Важно для разработчика: перед началом работы подключиться по VPN, проверить доступность `http://10.66.66.21:8080/api/channels` и убедиться, что на нужных каналах включён модуль **«Распознавание автономеров (Complete)»** (`platescomplete`).

Чеклист предварительной настройки Macroscop

Выполнить **до** разработки интеграции (через RDP или REST API):

- ☐ VPN подключён, сервер `10.66.66.21` доступен с машины/контейнера, где будет работать сервис интеграции.
- ☐ Пользователь API Macroscop входит в группу **«Старшие администраторы»** (требование документации для `configure`-запросов).
- ☐ `GET /api/channels` возвращает список камер КПП.
- ☐ Для каждой камеры КПП: `GET /configure/channels/{channel_id}/platescomplete` → `"Enabled": true`.
- ☐ В настройках `platescomplete` включено определение направления (`UseDirection: true`), настроены зоны/направления «Въезд» и «Выезд».
- ☐ Тестовый проезд машины генерирует событие `EventDescription: "Обнаружен автономер"` (проверить через `/event` или `/archive_events`).
- ☐ Зафиксировать `channel_id` камер въезда и выезда — они понадобятся для маппинга в Steiza.

Справочник Macroscop API (релевантные эндпоинты)

Авторизация

Macroscop поддерживает два способа (см. PDF, стр. 19–20):

1. HTTP Basic authentication (рекомендуется для сервиса интеграции):

```
Authorization: Basic Base64(login:password)
```

Для внутреннего пользователя MC пароль передаётся как **MD5-хэш** (не plaintext).

Пример для пользователя User с паролем 123 :

- MD5(123) = 202cb962ac59075b964b154e224b35d
- Строка для Base64: User:202cb962ac59075b964b154e224b35d
- Заголовок: Authorization: Basic
VXNlcjoyMDJjYjk2MmFjNTkwNzViOTY0YjE1NGUyMjRiMzVm

2. GET-параметры (альтернатива, подходит для HTTP API /event):

```
?login=User&password=202cb962ac59075b964b154e224b35d
```

Учётные данные Macroscop API **не хранить в репозитории**. Использовать переменные окружения / Django settings / secrets manager.

REST API — информация о системе

Метод	URL	Назначение
GET	/api/channels	Список камер (Id, Name, Disabled)
GET	/api/channels/{channel_id}	Полная информация о камере
GET	/api/channels/{channel_id}/status	Статус камеры (битрейт, ошибки)
GET	/configure/channels/{channel_id}/platescomplete	Настройки модуля ANPR (проверка Enabled)
GET	/api/platescomplete/allowedfeatures	Доступные шаблоны номеров по странам

Пример запроса списка камер:

```
GET http://10.66.66.21:8080/api/channels
Authorization: Basic VXNlcjoyMDJjYjk2MmFjNTkwNzViOTY0YjE1NGUyMjRiMzVm
```

Пример ответа (фрагмент):

```
[
  {
```

```

    "Id": "428d7aff-2e4a-46df-acff-0550cd827cd3",
    "Name": "КПП Въезд",
    "Disabled": false
  }
]

```

HTTP API — получение событий распознавания номеров

Вариант А: поток событий в реальном времени (рекомендуется как основной)

Долгоживущее HTTP-соединение к ресурсу `/event` :

```

GET http://10.66.66.21:8080/event?
login=User&password=202cb962ac59075b964b154e224b35d&responsetype=json&channelid={channel_id}&filter={event_type_id}

```

Особенности:

- Ответ — **NDJSON stream** (newline-delimited JSON) с `Transfer-Encoding: chunked`.
- Периодически приходят `KeepAlive` -сообщения (`InitiatorName: "System"`) — их нужно игнорировать.
- Параметры:
 - `channelid` — UUID камеры (опционально, для фильтрации по каналу).
 - `filter` — UUID типа события «Обнаружен автономер» (получить из `/archive_event_types`; в демо-режиме используется `00000000-0000-0000-0000-0000000000033`).
 - `responsetype=json` — формат ответа.
 - `mode=demo` — виртуальные события для отладки (не пишутся в журнал `Macroscop`).

Пример события «Обнаружен автономер»:

```

{
  "InitiatorName": "ExternalEvent",
  "EventId": "c9d6d086-c965-4cf8-ae6-85b3894e3a4a",
  "EventType": "Info",
  "EventDescription": "Обнаружен автономер",
  "ChannelId": "7d53f293-70d9-45df-9243-afe5fac19c65",
  "ChannelName": "КПП Въезд",
  "Timestamp": "26.03.2024 11:23:22",
  "ZonedTimestamp": "26.03.2024 11:23:22.200 +05:00",

```

```

    "Numberplate": "A123BC77",
    "plateText": "A123BC77",
    "Reliability": "0,9999972581863403",
    "direction": "Въезд",
    "RecognizedCarBrand": "Toyota",
    "RecognizedCarColors": "Желтый",
    "RecognizedCarType": "Легковой"
  }

```

Ключевые поля для Steiza:

Поле Macroscop	Назначение в Steiza
EventId	Идемпотентность (уникальный ключ события)
Numberplate / plateText	Госномер (нормализовать в upper case)
ZonedTimestamp	Время события (предпочтительно над Timestamp)
ChannelId	Определение камеры → тип события (въезд/выезд)
direction	Направление («Въезд» / «Выезд») — дублирует/уточняет маппинг камеры
Reliability	Достоверность распознавания (фильтрация шума)
RecognizedCarBrand , RecognizedCarColors , RecognizedCarType	Доп. метаданные (опционально сохранять)

Вариант В: опрос архива событий (fallback / восстановление пропусков)

```

POST http://10.66.66.21:8080/archive_events?
login=User&password=202cb962ac59075b964b154e224b35d
Content-Type: application/json

```

```

{
  "startTimeUtc": "24.06.2026 00:00:00.000",
  "endTimeUtc": "24.06.2026 23:59:59.999",
  "eventIds": ["<uuid-типа-события-Обнаружен-автономер>"],
  "channelIds": ["<uuid-камеры-въезда>", "<uuid-камеры-выезда>"],

```

```
"searchLimitCount": 1000
}
```

Использовать для:

- первичного backfill при запуске интеграции;
- восстановления после простоя сервиса (по курсору `last_processed_at`);
- периодического reconcile (Celery beat, раз в N минут).

Справочник типов событий

```
GET http://10.66.66.21:8080/archive_event_types?
login=User&password=...&responsetype=json
```

Возвращает статичные UUID типов событий — найти запись с `Name`, содержащим «автономер».

Техническая реализация

Общий подход

Задача состоит из **двух логических частей**:

1. **Django app `vehicle_fleet`** (или `autopark`) — доменные модели, REST API Steiza, бизнес-логика сопоставления номеров с автопарком.
2. **Сервис интеграции Macroscop** — отдельный долгоживущий процесс (или Celery worker), который:
 - подключается к Macroscop по VPN;
 - читает поток событий `/event` (основной режим);
 - периодически сверяет архив `/archive_events` (fallback);
 - создаёт записи `VehicleGatePass` в БД Steiza.

Backend — Django app `vehicle_fleet`

Следовать skill [add-django-api](#): `urls` → `views` → `serializers` → `services` → `models`.

Структура файлов

```
server/vehicle_fleet/
├─ __init__.py
├─ apps.py
```

```

├─ constants.py          # GatePassDirection, GatePassSource, CameraRole
├─ models.py
├─ serializers.py
├─ views.py
├─ urls.py
├─ services.py          # FleetVehicleService, GatePassService,
PlateNormalizer
├─ managers.py
├─ admin.py
├─ migrations/
├─ tests/
│   └─ test_services.py
│   └─ test_api_fleet.py
│   └─ test_api_gate_passes.py

server/vehicle_fleet_integration/
├─ __init__.py
├─ apps.py
├─ client.py            # MacroscopHttpClient (auth, channels, events)
├─ event_parser.py      # парсинг NDJSON, маппинг полей
├─ event_processor.py   # дедупликация, создание GatePass
├─ stream_worker.py     # long-poll loop /event
├─ archive_poller.py    # fallback poll /archive_events
├─ management/
│   └─ commands/
│       └─ run_macroscop_integration.py
├─ tests/
│   └─ test_event_parser.py
│   └─ test_event_processor.py
│   └─ fixtures/
│       └─ plate_event.json

```

Почему два app: `vehicle_fleet` — домен Steiza (API, модели, права).

`vehicle_fleet_integration` — инфраструктурный слой, изолированный от HTTP API Steiza; его можно запускать как management command или отдельный systemd/Celery process.

Подключение в проект

- Добавить оба app в `INSTALLED_APPS` (`server/server/settings.py`).
- Подключить URLs: `path("api/terminals/<int:terminal_pk>/", include("vehicle_fleet.urls"))`.
- Добавить настройки (см. раздел «Конфигурация»).

- Зарегистрировать новые actions в permissions терминала (аналогично entrypass_*).

Backend — сервис интеграции Macroscop

Ответственность разработчика

Компонент	Что делает
MacroscopHttpClient	HTTP-клиент с Basic auth / query auth, retry, timeout, логирование
stream_worker	Держит long-poll соединение к /event , читает NDJSON построчно, переподключается при обрыве
archive_poller	По расписанию (или при старте) запрашивает /archive_events за интервал [last_cursor, now]
event_parser	Парсит JSON события, отбрасывает KeepAlive/System, валидирует обязательные поля
event_processor	Нормализует номер, проверяет EventId на дубликат, определяет direction, создаёт VehicleGatePass
run_macroscop_integration	Management command / daemon endpoint

Алгоритм обработки события

1. Получить JSON-событие с EventDescription == "Обнаружен автономер".
2. Проверить Reliability >= MIN_RELIABILITY (настраиваемый порог, default 0.85).
3. Нормализовать Numberplate: upper case, убрать пробелы/дефисы (сохранить кириллицу).
4. Проверить идемпотентность: если macroscop_event_id уже есть в БД — пропустить.
5. Определить direction:
 - приоритет 1: поле direction из события («Въезд» → entry, «Выезд» → exit);
 - приоритет 2: маппинг GateCamera.role по ChannelId.
6. Найти FleetVehicle по registration_number + terminal_id (опциональная связь).
7. Создать VehicleGatePass со source=macroscop.
8. Обновить курсор MacroscopIntegrationState.last_event_at.

Дедупликация

Macroscop может генерировать повторные события для одного номера (параметр TimeSecNotRecognizeSamePlate в platescomplete). Стратегия:

- **Жёсткая:** уникальный индекс на `macroscop_event_id`.
- **Мягкая (дополнительно):** не создавать новый pass, если за последние `DEDUP_WINDOW_SEC` (default 60) уже есть запись с тем же номером, `direction` и `channel`.

Обработка ошибок и устойчивость

- При обрыве stream — exponential backoff reconnect (1s → 2s → 4s → max 60s).
- При недоступности VPN/Macroscop — логировать, не падать; продолжать retry.
- При ошибке записи в БД — сохранять событие в `MacroscopIntegrationState.pending_events` (JSON queue) или dead-letter таблицу для ручного replay.
- Healthcheck endpoint (опционально): `GET /api/terminals/{id}/macroscop/status` — `last_event_at`, `connected`, `error_message`.

Запуск в production

Варианты (выбрать один на этапе реализации, зафиксировать в README app):

1. **Management command** в отдельном systemd unit / Docker container:

```
python manage.py run_macroscop_integration --terminal-id=<pk>
```

2. **Celery long-running task** (менее предпочтительно для бесконечного stream).
3. **Celery beat** только для `archive_poller` (если stream нестабилен в инфраструктуре).

Рекомендация: **отдельный процесс** (вариант 1), т.к. `/event` — бесконечное соединение.

API Steiza (спроектированные эндпоинты)

Все эндпоинты — в контексте терминала, permissions через `TerminalActionBasedPermission`.

Автопарк (FleetVehicle)

Метод	Путь	Описание
GET	<code>/api/terminals/{terminal_pk}/fleet-vehicles/</code>	Список машин автопарка (фильтры: <code>search</code> , <code>is_active</code>)
POST	<code>/api/terminals/{terminal_pk}/fleet-vehicles/</code>	Добавить машину
GET	<code>/api/terminals/{terminal_pk}/fleet-vehicles/{id}/</code>	Карточка машины
PATCH	<code>/api/terminals/{terminal_pk}/fleet-</code>	Редактировать

Метод	Путь	Описание
	vehicles/{id}/	
DELETE	/api/terminals/{terminal_pk}/fleet-vehicles/{id}/	Удалить (soft delete через is_active=false предпочтительнее)

POST body (пример):

```
{
  "registration_number": "A123BC77",
  "brand": "КамАЗ",
  "model": "54901",
  "comment": "Тягач терминала",
  "is_active": true
}
```

Response 201:

```
{
  "id": 1,
  "registration_number": "A123BC77",
  "brand": "КамАЗ",
  "model": "54901",
  "comment": "Тягач терминала",
  "is_active": true,
  "created_at": "2026-06-24T10:00:00+03:00",
  "updated_at": "2026-06-24T10:00:00+03:00"
}
```

Журнал въездов/выездов (VehicleGatePass)

Метод	Путь	Описание
GET	/api/terminals/{terminal_pk}/vehicle-gate-passes/	Журнал (фильтры: date_from, date_to, direction, registration_number, fleet_vehicle_id, source)
GET	/api/terminals/{terminal_pk}/vehicle-gate-passes/{id}/	Детали записи
POST	/api/terminals/{terminal_pk}/vehicle-gate-passes/	Ручная запись (source=manual, для корректировок диспетчером)

Автоматические записи (source=macroscop) создаёт только сервис интеграции, не через публичный API.

GET response (фрагмент списка):

```
{
  "count": 2,
  "results": [
    {
      "id": 101,
      "registration_number": "A123BC77",
      "direction": "entry",
      "passed_at": "2026-06-24T08:15:22+03:00",
      "source": "macroscop",
      "fleet_vehicle": { "id": 1, "registration_number": "A123BC77" },
      "gate_camera": { "id": 1, "name": "КПП Въезд", "role": "entry" },
      "reliability": 0.999,
      "recognized_brand": "Toyota",
      "macroscop_event_id": "c9d6d086-c965-4cf8-aef6-85b3894e3a4a"
    }
  ]
}
```

Настройки камер (GateCamera) — admin/internal API

Метод	Путь	Описание
GET	/api/terminals/{terminal_pk}/gate-cameras/	Список привязанных камер
POST	/api/terminals/{terminal_pk}/gate-cameras/	Привязать камеру Macroscop к терминалу
PATCH	/api/terminals/{terminal_pk}/gate-cameras/{id}/	Обновить role/название
POST	/api/terminals/{terminal_pk}/gate-cameras/sync/	Подтянуть список камер из Macroscop /api/channels

Статус интеграции (опционально, v1.1)

Метод	Путь	Описание
GET	/api/terminals/{terminal_pk}/macroscop-integration/status/	last_event_at, is_connected, last_error

Frontend (tos-client)

Минимальный scope для v1 (при наличии ресурса; иначе — только API + admin):

1. **Страница «Автопарк»** — таблица `FleetVehicle`, форма создания/редактирования.
2. **Страница «Журнал въездов/выездов»** — таблица `VehicleGatePass` с фильтрами по дате, номеру, направлению.
3. **Настройки интеграции** (для администратора) — список `GateCamera`, кнопка sync.

Следовать MVS в `tos-client/` (skill [add-tos-page](#)):

```
tos-client/src/modules/vehicle-fleet/  
├─ list/           # список автопарка  
├─ gate-passes/    # журнал въездов/выездов  
└─ shared/         # типы, API hooks (TanStack Query)
```

Связь с T-031: при выборе транспорта в заявках/пропусках — приоритет машин из `FleetVehicle` (можно вынести в follow-up, если не входит в scope v1).

Конфигурация (environment / settings)

```
# server/server/settings.py (или отдельный settings_local)  
  
MACROSCOP_INTEGRATION = {  
    "ENABLED": env.bool("MACROSCOP_ENABLED", default=False),  
    "BASE_URL": env("MACROSCOP_BASE_URL",  
default="http://10.66.66.21:8080"),  
    "LOGIN": env("MACROSCOP_LOGIN", default="User"),  
    "PASSWORD_MD5": env("MACROSCOP_PASSWORD_MD5", default=""), # MD5 от  
    пароля  
    "TERMINAL_ID": env.int("MACROSCOP_TERMINAL_ID", default=None),  
    "MIN_RELIABILITY": env.float("MACROSCOP_MIN_RELIABILITY", default=0.85),  
    "DEDUP_WINDOW_SEC": env.int("MACROSCOP_DEDUP_WINDOW_SEC", default=60),  
    "ARCHIVE_POLL_INTERVAL_SEC": env.int("MACROSCOP_ARCHIVE_POLL_INTERVAL",  
default=300),  
    "REQUEST_TIMEOUT_SEC": env.int("MACROSCOP_REQUEST_TIMEOUT", default=30),  
}
```

Permissions (новые actions)

Предлагаемые actions для `terminal.CustomGroup`:

Action	Кто	Описание
fleet_vehicle_view	Диспетчер, админ	Просмотр автопарка и журнала
fleet_vehicle_edit	Админ терминала	CRUD автопарка, ручные записи в журнале
gate_camera_manage	Админ терминала	Настройка привязки камер

Модели и связи

```
# server/vehicle_fleet/models.py

class FleetVehicle(TimeStampVisibleModel):
    """
    Машина автопарка терминала.
    Логика: справочник «своих» машин; при совпадении номера в событии
    Macroscop
    запись журнала ссылается на эту сущность.
    """
    terminal = models.ForeignKey('terminal.Terminal',
on_delete=models.CASCADE, related_name='fleet_vehicles')
    registration_number = models.CharField(max_length=32) #
нормализованный, upper
    brand = models.CharField(max_length=128, blank=True, default='')
    model = models.CharField(max_length=128, blank=True, default='')
    comment = models.TextField(blank=True, default='')
    is_active = models.BooleanField(default=True)

    class Meta:
        unique_together = [('terminal', 'registration_number')]

class GateCamera(TimeStampVisibleModel):
    """
    Привязка канала Macroscop к терминалу.
    Логика: ChannelId из Macroscop → роль (въезд/выезд) для определения
    direction,
    если поле direction в событии отсутствует или неоднозначно.
    """
    terminal = models.ForeignKey('terminal.Terminal',
on_delete=models.CASCADE, related_name='gate_cameras')
    macroscop_channel_id = models.UUIDField() # ChannelId из /api/channels
    name = models.CharField(max_length=255) # ChannelName, для UI
```

```

        role = models.CharField(max_length=16, choices=GateCameraRole.CH0ICES)
# entry | exit
        is_active = models.BooleanField(default=True)

        class Meta:
            unique_together = [('terminal', 'macroscop_channel_id')]

class VehicleGatePass(TimeStampVisibleModel):
    """
    Запись журнала въезда/выезда.
    Логика: одна строка = один факт проезда через КПП.
    """
    terminal = models.ForeignKey('terminal.Terminal',
on_delete=models.CASCADE, related_name='vehicle_gate_passes')
    fleet_vehicle = models.ForeignKey(FleetVehicle, null=True, blank=True,
on_delete=models.SET_NULL)
    gate_camera = models.ForeignKey(GateCamera, null=True, blank=True,
on_delete=models.SET_NULL)
    registration_number = models.CharField(max_length=32)
    direction = models.CharField(max_length=16,
choices=GatePassDirection.CH0ICES) # entry | exit
    passed_at = models.DateTimeField() # время из ZonedDateTime
    source = models.CharField(max_length=16, choices=GatePassSource.CH0ICES)
# macroscop | manual
    reliability = models.FloatField(null=True, blank=True)
    recognized_brand = models.CharField(max_length=128, blank=True,
default='')
    recognized_color = models.CharField(max_length=64, blank=True,
default='')
    recognized_type = models.CharField(max_length=64, blank=True,
default='')
    macroscop_event_id = models.UUIDField(null=True, blank=True,
unique=True)
    raw_event = models.JSONField(default=dict) # полный payload для аудита
    created_by = models.ForeignKey('accounts.CustomUser', null=True,
blank=True, on_delete=models.SET_NULL)

class MacroscopIntegrationState(models.Model):
    """
    Singleton на терминал: курсор интеграции, статус подключения.
    """
    terminal = models.OneToOneField('terminal.Terminal',
on_delete=models.CASCADE, related_name='macroscop_state')
    last_event_at = models.DateTimeField(null=True, blank=True)

```

```
last_archive_poll_at = models.DateTimeField(null=True, blank=True)
is_connected = models.BooleanField(default=False)
last_error = models.TextField(blank=True, default='')
updated_at = models.DateTimeField(auto_now=True)
```

Связь с существующими моделями:

- `transport.Transport` — глобальный справочник ТС (не заменяем; `FleetVehicle` — терминальный автопарк).
- `entrypass.EntryPass` — пропуска на ввоз/вывоз контейнеров; в v1 **не связываем** автоматически, но поле `registration_number` позволяет матчить вручную.
Автолинковка — отдельная follow-up задача.

Примеры данных и сценариев

Примеры мок-данных

```
# Создание машины автопарка
fleet_vehicle_payload = {
    "registration_number": "A123BC77",
    "brand": "КамАЗ",
    "model": "54901",
    "comment": "Тягач",
    "is_active": True,
}

# Привязка камеры
gate_camera_payload = {
    "macroscop_channel_id": "7d53f293-70d9-45df-9243-afe5fac19c65",
    "name": "КПП Въезд",
    "role": "entry",
    "is_active": True,
}

# Событие Macroscop (fixture для тестов)
macroscop_plate_event = {
    "EventId": "c9d6d086-c965-4cf8-aef6-85b3894e3a4a",
    "EventDescription": "Обнаружен автономер",
    "ChannelId": "7d53f293-70d9-45df-9243-afe5fac19c65",
    "Numberplate": "A123BC77",
    "ZonedTimestamp": "26.03.2024 11:23:22.200 +05:00",
    "direction": "Въезд",
    "Reliability": "0,9999972581863403",
}
```

```
"RecognizedCarBrand": "Toyota",  
}
```

Примеры логических сценариев

Позитивные

1. Автоматический въезд машины из автопарка

- Вход: событие Macroscop с Numberplate=A123BC77 , direction=Въезд .
- В автопарке есть FleetVehicle(registration_number=A123BC77) .
- Ожидание: создаётся VehicleGatePass(direction=entry, source=macroscop, fleet_vehicle=<linked>) .

2. Выезд незнакомой машины

- Вход: событие с номером, которого нет в автопарке.
- Ожидание: VehicleGatePass создаётся, fleet_vehicle=null .

3. Ручное добавление машины в автопарк

- Вход: POST /fleet-vehicles/ с новым номером.
- Ожидание: 201, номер нормализован в upper case, уникален в рамках терминала.

4. Восстановление после простоя

- Вход: сервис был offline 2 часа; при старте archive_poller запрашивает /archive_events за пропущенный интервал.
- Ожидание: все пропущенные события созданы, дубликаты по macroscop_event_id отброшены.

Негативные

5. Дубликат события

- Вход: повторное событие с тем же EventId .
- Ожидание: запись не создаётся, warning в лог.

6. Низкая достоверность

- Вход: Reliability=0.3 .
- Ожидание: событие отброшено (не создаётся pass), debug-лог.

7. KeepAlive от Macroscop

- Вход: InitiatorName=System , пустой plateText .
- Ожидание: игнорируется, не пишется в БД.

8. Дубликат номера в автопарке

- Вход: POST с номером, который уже есть у терминала.
- Ожидание: 400 ValidationError.

Граничные

9. Событие без direction, но с известным ChannelId

- Ожидание: direction определяется по `GateCamera.role`.

10. Камера не привязана к терминалу

- Ожидание: событие логируется как warning; pass создаётся с `gate_camera=null` (или отбрасывается — зафиксировать в реализации, предпочтительно создавать с null для аудита).

Критерии приёмки

Функциональные

- ☐ Подключение к Macroscop по VPN, успешный `GET /api/channels`.
- ☐ Подтверждено: модуль `platescomplete` включён на камерах КПП.
- ☐ Сервис интеграции получает события «Обнаружен автономер» и создаёт записи журнала.
- ☐ CRUD автопарка работает через API Steiza.
- ☐ Журнал въездов/выездов отображает автоматические и ручные записи с фильтрами.
- ☐ Дедупликация по `macroscop_event_id` работает.
- ☐ После перезапуска сервиса пропущенные события подтягиваются из архива.

Тестовые сценарии

- ☐ Unit: нормализация госномера (`test_services.py`)
- ☐ Unit: парсинг NDJSON потока, фильтрация KeepAlive (`vehicle_fleet_integration/tests/test_event_parser.py`)
- ☐ Unit: дедупликация и создание pass (`test_event_processor.py`)
- ☐ Unit: определение direction по camera role и полю direction (`test_event_processor.py`)
- ☐ API: CRUD fleet-vehicles — успех и валидация дубликата (`test_api_fleet.py`)
- ☐ API: список gate-passes с фильтрами (`test_api_gate_passes.py`)
- ☐ API: permissions — диспетчер не может редактировать автопарк (`test_api_fleet.py`)
- ☐ Integration (mock): полный pipeline event JSON → VehicleGatePass в БД (`test_event_processor.py`)

Нефункциональные

- ☐ Секреты Macroscop не попадают в git.

- ☐ Логи интеграции содержат `event_id`, `registration_number`, `direction` (без паролей).
- ☐ Сервис переподключается к stream после обрыва соединения.

Открытые вопросы (уточнить у @ter224 до старта)

#	Вопрос	Влияние
1	Точный <code>terminal_id</code> / slug терминала в Steiza для ter224	Привязка моделей
2	UUID камер въезда и выезда	Маппинг <code>GateCamera</code>
3	Учётные данные Macroscop API (отдельно от RDP?)	Авторизация
4	Часовой пояс терминала	Парсинг <code>ZonedTimestamp</code>
5	Нужна ли автосвязь с <code>EntryPass</code> / заказами в v1	Scope frontend и моделей
6	Где физически запускается сервис интеграции (сервер Steiza / сервер терминала)	VPN routing, деплой
7	UUID типа события «Обнаружен автономер» на данном сервере	Параметр <code>filter</code> в <code>/event</code>

План работ (рекомендуемый порядок)

- Инфраструктура:** VPN, проверка Macroscop API, документирование `channel_id` и `event type id`.
- Модели + миграции:** `vehicle_fleet` app, admin для отладки.
- API автопарка и журнала:** CRUD, permissions, тесты.
- Клиент Macroscop:** auth, channels, event stream parser (с mock/fixtures).
- Сервис интеграции:** stream worker + archive poller + management command.
- E2E на терминале:** тестовый проезд, проверка записи в журнале.
- Frontend (опционально v1):** страницы автопарка и журнала в tos-client.

Артефакты задачи

Артефакт	Описание
macroscop-4.6-api.pdf	Официальная документация API
VPN .conf	Предоставляется заказчиком (не в репозитории)
Telegram @ter224	Контакт для доступов и настройки терминала